

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 1 de 24

1. TABLA DE CONTENIDO

1. TABLA DE CONTENIDO.....1

2. INTRODUCCIÓN2

3. OBJETIVO3

4. DEFINICIONES4

5. DESARROLLO5

 5.1 Resumen Ejecutivo.....5

 5.2 Arquitectura General6

 5.3 Backend Django.....6

 5.4 Frontend React9

 5.5 Integración Frontend - Backend 10

 5.6 Endpoints Encontrados 11

 5.7 Seguridad Observable..... 17

 5.8 Organización del Proyecto 18

 5.9 Observaciones Técnicas 19

 5.10 Conclusión..... 20

6. DOCUMENTOS DE REFERENCIA21

7. ANEXOS22

8. CONTROL DE CAMBIOS23

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 2 de 24

2. INTRODUCCIÓN

El presente documento corresponde al informe de auditoría técnica QA Diagnóstico realizado sobre el aplicativo IGED Móvil, en el marco del proyecto RNI - VIVANTO (Encuesta IGED Móvil 2026-04-1) de la Unidad para las Víctimas, cuyo objeto es la caracterización de información de víctimas del conflicto armado en el contexto de la Ley 1448 de 2011 y el Registro Nacional de Información (RNI).

La revisión se realizó en modo exclusivo de lectura, mediante inspección estática del código fuente, sin ejecución de la aplicación ni modificación de archivos del repositorio. El alcance auditado comprende el backend desarrollado en Django REST Framework (srni-backend/) y el frontend web desarrollado en React + Vite (srni-frontend/); la aplicación móvil (srni-mobile/) quedó fuera del alcance solicitado para esta revisión.

Toda afirmación contenida en el desarrollo de este informe está respaldada por lectura directa de archivos del repositorio, incluyendo referencias de archivo y línea cuando aplica. La fecha de corte del análisis corresponde entre el 13 y 17 de julio de 2026.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 3 de 24

3. OBJETIVO

Verificar el estado técnico del backend y del frontend del aplicativo IGED Móvil, mediante inspección estática de código fuente en modo solo lectura, con el fin de identificar riesgos de arquitectura, seguridad y calidad del software, para brindar al equipo técnico y de supervisión del proyecto información objetiva, verificable y trazable que respalde la toma de decisiones sobre el estado de avance y los pendientes del desarrollo.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 4 de 24

4. DEFINICIONES

RNI / VIVANTO: Registro Nacional de Información y sistema misional de la Unidad para las Víctimas para la caracterización y atención de víctimas del conflicto armado.

IGED Móvil: Aplicativo móvil/web objeto de esta auditoría, usado para la captura de información en campo bajo el proceso de caracterización de víctimas.

DRF (Django REST Framework): Framework sobre Django utilizado para construir la API REST del backend auditado.

JWT (JSON Web Token): Mecanismo de autenticación basado en tokens firmados, usado en el backend para autenticar y autorizar solicitudes.

PII (Personally Identifiable Information): Información personal identificable (nombres, número de documento, fecha de nacimiento, entre otros) sujeta a cifrado y control de acceso.

RBAC (Role-Based Access Control): Modelo de control de acceso basado en roles y permisos asociados a un perfil de usuario.

Repositorio Mock: Componente de software que simula el comportamiento de una fuente de datos real (en este caso, el repositorio de víctimas) con datos ficticios, usado mientras no se conecta la fuente definitiva.

SPA (Single Page Application): Aplicación web que carga una sola página HTML y actualiza su contenido dinámicamente, como es el caso del frontend en React.

Celery: Sistema de colas de tareas asíncronas usado en arquitecturas Django para procesos en segundo plano.

CORS / HSTS / CSP: Cabeceras y políticas de seguridad HTTP (control de origen cruzado, forzado de HTTPS y política de seguridad de contenido, respectivamente) usadas para proteger la aplicación en producción.

Fernet: Esquema de cifrado simétrico (AES en modo CBC combinado con HMAC-SHA256) empleado para el cifrado de campos PII en el backend.

Throttling: Mecanismo de limitación de tasa de peticiones a un endpoint, usado para mitigar abuso o ataques de fuerza bruta.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 5 de 24

5. DESARROLLO

A continuación se presenta el desarrollo completo de la auditoría técnica realizada, conservado tal como fue producido por el equipo de QA, organizado en las diez (10) secciones originales del informe de auditoría.

INFORME DE AUDITORÍA TÉCNICA — MODO SOLO LECTURA

Proyecto: RNI - VIVANTO - ENCUESTA IGED MÓVIL 2026-04-1 **Alcance auditado:** `srni-backend/` (Django REST Framework) y `srni-frontend/` (React + Vite) **Metodología:** Inspección estática de código fuente, sin ejecución, sin modificación de archivos. **Fecha de corte del análisis:** 2026-07-15

Nota metodológica: toda afirmación de este informe está respaldada por lectura directa de archivos del repositorio (con referencia `archivo:línea` cuando aplica).

1. RESUMEN EJECUTIVO

El repositorio corresponde a una aplicación de captura de información para caracterización de víctimas (contexto Ley 1448 / RNI, con referencias a UARIV), compuesta por tres frentes de código: un backend Django REST Framework (`srni-backend/`), un frontend web React (`srni-frontend/`) y una aplicación móvil (`srni-mobile/`, fuera del alcance solicitado de esta auditoría, que se limitó a backend y frontend web).

El backend implementa 10 aplicaciones Django funcionales (autenticación JWT, víctimas con cifrado de PII, motor de formularios dinámico, hogares, encuestas, catálogos paramétricos, un asistente de IA, auditoría, distribución de APK y reportes de producción) más una aplicación adicional (`sincronizacion`) registrada pero sin ningún contenido implementado. El frontend es una SPA React con enrutamiento protegido por permisos, cliente HTTP centralizado con manejo de JWT y refresco automático de token, y una capa de estado mínima (un único store de autenticación).

El hallazgo más relevante a nivel de arquitectura es que la fuente de datos de víctimas (`VICTIMA_REPOSITORY`) **no está configurada en ningún entorno**, por lo que el sistema siempre opera con un repositorio simulado (`MockVictimaRepository`) con datos ficticios, incluso en la configuración de producción. Esto debe confirmarse con el equipo de desarrollo como estado esperado de la fase actual del proyecto o como pendiente de integración con la fuente real (Oracle/RUV).

No se identificaron señales de código malicioso, backdoors, ni prácticas de exfiltración de datos. Las prácticas de seguridad observadas (JWT con rotación y blacklist, cifrado de campos PII, contraseñas con Argon2, throttling, HSTS/CSP en producción) son consistentes con un proyecto tratado con seriedad en materia de protección de datos personales, propio del manejo de información de víctimas del conflicto armado.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 6 de 24

2. ARQUITECTURA GENERAL

Patrón general: arquitectura cliente-servidor desacoplada — backend API REST (Django REST Framework) consumido por un frontend SPA (React) vía JSON sobre HTTP, y adicionalmente por una app móvil (fuera de alcance).

Backend: arquitectura por aplicaciones Django (app-per-domain), cada una con su propio `models.py`, `views.py`, `serializers.py`, `urls.py` y, en algunos casos, `permissions.py/validators.py/filters.py` propios. Autenticación centralizada vía JWT (SimpleJWT), permisos basados en un modelo de Perfil con flags booleanos (RBAC simple, no granular por objeto salvo excepciones puntuales).

Frontend: SPA con Vite + React Router, un único store global (Zustand) para sesión/autenticación, resto del estado manejado localmente por página. Cliente HTTP centralizado (Axios) con interceptores para autenticación y errores.

Infraestructura (docker-compose.yml, infra/): contenedores para `nginx` (proxy/TLS), `backend` (gunicorn), `celery` (worker), `postgres` y `redis`, con secretos de Postgres gestionados vía Docker secrets (archivos, no variables planas) — buena práctica observada. Redis se usa tanto para caché de Django como para broker aparente de Celery (aunque, ver sección 9, no se halló código de tareas Celery).

Separación de responsabilidades: clara a nivel de capas (modelos/serializers/vistas/urls separados), aunque con duplicación de utilidades transversales (extracción de IP repetida en 6 archivos distintos en vez de un middleware o helper compartido) y sin capa de "servicios" formal en la mayoría de apps (la lógica de negocio vive mayormente en las vistas, con la excepción de `apps/victimas/repository/` que sí implementa un patrón Strategy/Repository, y `apps/ia/services.py`).

Clasificación: 🟡 **Medio** — la arquitectura general es coherente y modular, pero existen elementos de infraestructura declarados sin implementación real (Celery, app `sincronizacion`) que introducen ambigüedad sobre el estado real del sistema frente a lo que la configuración sugiere.

3. BACKEND DJANGO

3.1 Configuración (`srni/settings/`)

- Archivos: `base.py`, `development.py`, `production.py`, `servidor.py` (este último hereda de `production.py` y **desactiva** `SECURE_SSL_REDIRECT`, cookies seguras y HSTS — documentado en comentarios como fase temporal de despliegue por IP sin TLS).
- `INSTALLED_APPS` (`base.py:22-56`): Unfold (admin), apps estándar de Django, `rest_framework`, `rest_framework_simplejwt` (+ `token_blacklist`), `corsheaders`, `django_filter`, `drf_spectacular`, y las 11 apps locales del dominio.
- `MIDDLEWARE` (`base.py:192-202`): stack estándar de Django + Whitenoise + CORS, sin middleware custom del proyecto.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 7 de 24

- `REST_FRAMEWORK` (`base.py:240-266`): autenticación JWT única, permiso por defecto `IsAuthenticated` (todo cerrado salvo excepción explícita), paginación por número de página (20 ítems), throttling diferenciado (`anon`, `user`, `login=5/min`, `busqueda_rni=30/h`, `ia_consulta=20/h`).
- `SECRET_KEY` y credenciales de BD: desde variables de entorno (`python-decouple`), sin valores por defecto hardcoded (`base.py:16`).
- Base de datos: SQLite en desarrollo, PostgreSQL (`psycopg v3`) en producción, con `sslmode` configurable.
- Password hashers: Argon2 primero, luego PBKDF2 (`base.py:289-300`), validador de longitud mínima 10.
- Cifrado de PII: `FIELD_ENCRYPTION_KEY` desde entorno, sin valor por defecto operativo (falla explícitamente si no se configura).

3.2 URLs raíz (`srni/urls.py`)

Estructura de árbol de rutas bajo `/api/`, más `/admin/`, `/health/`, `/api/docs/` (Swagger), `/api/redoc/`, `/api/schema/`. Detalle completo en sección 6.

3.3 Aplicaciones Django

App	Propósito observado	Estado
<code>autenticacion</code>	Login JWT, usuarios, perfiles, permisos RBAC	Completa
<code>victimas</code>	Registro/búsqueda de víctimas con PII cifrada, repositorio externo (mock)	Completa, pero fuente de datos real no configurada
<code>formulario</code>	Motor de formularios dinámico (instrumentos, capítulos, preguntas, skip-logic)	Completa
<code>hogares</code>	Gestión de hogares y miembros	Completa
<code>encuestas</code>	Sesiones de encuesta y respuestas	Completa
<code>parametricas</code>	Catálogos geográficos/étnicos de solo lectura	Completa
<code>Ia</code>	Proxy a Gemini para asistente de voz	Completa
<code>auditoria</code>	Log de acceso inmutable	Completa
<code>Móvil</code>	Distribución de APK	Completa (mínima)
<code>reportes</code>	Reportes de producción/supervisión	Completa, sin modelos propios
<code>sincronizacion</code>	Registrada en <code>INSTALLED_APPS</code> , sin <code>include()</code> en <code>urls</code>	Vacía / sin implementar

Detalle de modelos, permisos y endpoints por app: ver sección 6.

3.4 Middleware, autenticación y permisos

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 8 de 24

- No existe middleware custom del proyecto — verificado por ausencia de archivos `middleware.py` en `srni/` y en todas las apps.
- Autenticación: exclusivamente JWT (`rest_framework_simplejwt`), sin `SessionAuthentication` en el stack de DRF.
- Permisos custom por perfil: `PuedeAdministrar`, `PuedeBuscarRNI`, `PuedeCaracterizar`, `PuedeVerReportes`, `PuedeConsultarOperacion` (`apps/autenticacion/permissions.py`), todos delegando en `Usuario.puede(accion)`.
- `AllowAny` usado únicamente en: `api_root`, `log_error_mobile` (solo si `DEBUG=True`), `LoginView`, y los dos endpoints de `apps/movil` (versión/descarga de APK) — todos justificables por ser públicos o previos a autenticación.

3.5 Validaciones

- Validaciones a nivel de serializer (`validate_*`, `validate`) presentes en casi todas las apps.
- `HogarValidator` (`apps/hogares/validators.py`) implementa reglas de negocio (tutor requiere menores, cuidador requiere dependiente, autorizado válido) pero **no se encontró invocación desde las vistas HTTP** — solo se ejerce desde tests unitarios directos. Requiere confirmación del equipo de desarrollo sobre si es código pendiente de conectar o deuda intencional.
- Evaluador de expresiones de skip-logic (`apps/formulario/views.py:32-96`) usa un AST restringido en lugar de `eval()` — buena práctica de seguridad de entrada.

3.6 Dependencias (`requirements.txt`)

Django 5.2.15, DRF 3.15.2, SimpleJWT 5.4.0, drf-spectacular, django-cors-headers, django-filter, django-unfold, psycpg 3, cryptography, argon2-cffi, redis, celery, django-redis, boto3/django-storages, python-decouple, google-generativeai, gunicorn/whitenoise, pytest/pytest-django/factory-boy.

Dependencias declaradas sin uso detectado en el código: `django-auditlog` (no está en `INSTALLED_APPS` ni se importa; la app tiene su propio sistema de auditoría manual) y `django-ratelimit` (no hay ninguna referencia fuera de `requirements.txt`).

No se encontró SDK de rastreo de errores (Sentry u otro) en dependencias ni en código.

3.7 Tests

Existen suites de pytest en `srni-backend/tests/` (encuestas, formulario, hogares, IA, paramétricas, víctimas) y tests específicos por app en `apps/*/tests/` (permisos por rol, carga de diccionario, constancias de hogar, validadores, padrón offline). No se hallaron tests para `apps/parametricas` fuera del archivo raíz, ni para `auditoria`, `movil`, `reportes`, `sincronizacion` (esta última no tiene código que testear).

Semaforización Backend: ● **Medio**

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 9 de 24

Justificación: la estructura por app es clara, las prácticas de seguridad (JWT, cifrado PII, throttling, permisos) están bien implementadas y documentadas en el propio código. Se clasifica en Medio y no en Bajo por: (a) el repositorio de víctimas real no está configurado en ningún entorno, (b) existe una app fantasma (*sincronizacion*) que introduce confusión sobre el alcance real del sistema, (c) infraestructura Celery declarada sin tareas implementadas, y (d) dependencias declaradas sin uso. Ninguno de estos hallazgos constituye una falla de seguridad crítica evidente, por lo que no se clasifica como Crítico.

4. FRONTEND REACT

4.1 Stack y configuración

- React 18.3, React Router 6.23, Zustand 4.5 (estado), Axios 1.7 (HTTP), React Hook Form + Zod (formularios, uso parcial), Recharts/react-simple-maps (visualización), Sonner (notificaciones), Vite 5.3 + TypeScript 5.4 estricto.
- `vite.config.ts`: proxy de `/api` hacia el backend (`VITE_API_URL`, default `http://localhost:8001`), alias `@` → `/src`.
- `.env.example`: variable `VITE_API_URL` como única configuración de entorno relevante para la integración.

4.2 Enrutamiento

Todas las rutas se definen en `src/App.tsx`, sin archivo de rutas separado. Ruta pública única: `/login`. El resto de rutas están anidadas bajo un componente `RequireAuth` que valida presencia de token en el store y carga el perfil del usuario. Dos rutas adicionales (`/supervision`, `/auditoria`, `/usuarios`) están protegidas además por un componente `RequirePermission` que verifica permisos específicos del perfil (`puede_ver_reportes`, `puede_administrar`, o rol `COORDINADOR`/`ADMINISTRADOR`). La mayoría de páginas usa `React.lazy` para carga diferida.

4.3 Autenticación en frontend

- Tokens (`access_token`, `refresh_token`) almacenados en `sessionStorage`, explícitamente no en `localStorage` (decisión documentada en el propio código, `src/stores/authStore.ts:3-8`).
- Interceptor de request adjunta `Authorization: Bearer <token>` automáticamente (`src/api/client.ts:20-26`).
- Interceptor de response maneja 401 con refresco automático de token, cola de peticiones pendientes durante el refresh, y logout + redirección a `/login` si el refresh falla (`src/api/client.ts:32-93`).
- Logout limpia todo `sessionStorage` (no solo los tokens) y notifica al backend antes de limpiar el estado local.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 10 de 24

4.4 Capa API y estado

Un cliente Axios centralizado (`src/api/client.ts`) del cual dependen 10 módulos especializados por dominio (`auth`, `auditoria`, `encuestas`, `formulario`, `hogares`, `parametricas`, `reportes`, `supervision`, `usuarios`, `victimas`). Un único store global de estado (Zustand) dedicado exclusivamente a autenticación; el resto de páginas maneja su propio estado local.

4.5 Páginas y componentes

17 páginas identificadas, cada una mapeada a los módulos de API correspondientes (detalle completo entregado por el módulo de investigación, disponible bajo solicitud si se requiere el listado línea por línea). Componentes organizados en `layout/shell` (`MainLayout`, `Sidebar`), manejo de errores (`ErrorBoundary`) y una biblioteca propia de 14 componentes UI (`src/components/ui/`) sin dependencia de un kit externo (Tailwind + design system propio "gov").

4.6 Manejo de errores

Error Boundary de React envolviendo toda la aplicación (`src/main.tsx`), más un interceptor global en el cliente Axios que traduce errores de red, 5xx y 403 a notificaciones toast (`sonner`). El caso 401 se gestiona aparte mediante el flujo de refresh de token.

4.7 Formularios

React Hook Form + Zod están instalados pero **solo se usan en una pantalla** (`CambiarPassword.tsx`); el resto de formularios (login, gestión de usuarios, filtros) usa `useState` manual sin librería de validación.

4.8 Tests

Cobertura mínima: un test de store (`authStore.test.ts`) y un test de componente UI (`Button.test.tsx`). No hay tests de páginas, del cliente API/interceptores, de rutas/guards (`App.tsx`) ni del `ErrorBoundary`.

Semaforización Frontend: ● Bajo / ● Medio (mixto)

Justificación: la estructura de enrutamiento, autenticación y manejo de errores está bien organizada y sigue prácticas consistentes (token en `sessionStorage`, interceptores centralizados, guards de ruta). Se matiza a Medio únicamente por la cobertura de tests casi inexistente y el uso inconsistente de la librería de formularios instalada (adoptada en una sola pantalla). No se identifican riesgos estructurales relevantes.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 11 de 24

5. INTEGRACIÓN FRONTEND - BACKEND

Comunicación: HTTP/JSON vía Axios contra el backend Django, con proxy de desarrollo (`vite.config.ts`) redirigiendo `/api/*` al puerto 8001, y `VITE_API_URL` como variable de entorno para apuntar a distintos backends (local, red local, producción).

Autenticación: flujo estándar OAuth2-like con JWT: `POST /api/auth/token/` obtiene `access/refresh`; el frontend adjunta el `access` en cada petición; ante un 401, intenta automáticamente `POST /api/auth/token/refresh/` y reintenta la petición original; si el `refresh` falla, cierra sesión.

Intercambio de información: el frontend consume los endpoints del backend descritos en la sección 6 mediante módulos por dominio, sin capa de transformación intermedia relevante (los tipos TypeScript reflejan directamente los serializers del backend, según los nombres de campos observados).

Manejo de errores: centralizado en el interceptor de Axios del frontend (mapea códigos HTTP a mensajes de usuario); el backend responde con los códigos y estructuras estándar de DRF (no se verificó un formato de error custom unificado más allá de lo que DRF entrega por defecto).

Observación de consistencia: el endpoint `GET /api/parametricas/tipos-documento/` es consumido desde dos módulos distintos del frontend (`parametricas.ts` y `victimas.ts`, este último con filtro `activo=true`) — mismo recurso, dos puntos de acceso en el código frontend. No representa un problema de integración, pero sí una oportunidad de consolidación.

Semaforización Integración: ● Bajo

Justificación: el flujo de autenticación, refresco de token y consumo de API está implementado de forma coherente y verificable en ambos extremos, sin discrepancias evidentes entre lo que el frontend espera y lo que el backend expone.

6. ENDPOINTS ENCONTRADOS

Autenticación: salvo indicación contraria, todos los endpoints requieren JWT (`Authorization: Bearer`) y `IsAuthenticated` como mínimo.

6.1 Raíz / sistema

Método	URL	Vista	Auth	Finalidad
GET	<code>/health/</code>	<code>health_check</code>	Ninguna	Health check
GET	<code>/api/</code>	<code>api_root</code>	<code>AllowAny</code>	Índice de API
GET	<code>/admin/</code>	Django Admin (Unfold)	Sesión Django	Panel administrativo

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 12 de 24

Método	URL	Vista	Auth	Finalidad
GET	/api/schema/, /api/docs/, /api/redoc/	drf-spectacular	Según config	Documentación OpenAPI
GET	/api/_debug/log/	log_error_mobile	AllowAny, solo si DEBUG=True	Logging de errores móviles en desarrollo

6.2 apps.autenticacion — /api/auth/ y /api/usuarios/

Método	URL	Vista	Auth	Permiso	Finalidad
POST	/api/auth/login/ (y alias /token/)	LoginView	AllowAny	—	Login, emite JWT
POST	/api/auth/refresh/ (y alias /token/refresh/)	TokenRefreshView	AllowAny	—	Refresco de access token
POST	/api/auth/logout/	LogoutView	JWT	IsAuthenticated	Blacklist de refresh token
GET	/api/auth/me/ (y alias /perfil/)	MeView	JWT	IsAuthenticated	Perfil del usuario autenticado
POST	/api/auth/cambiar-password/	CambiarPasswordView	JWT	IsAuthenticated	Cambio de contraseña propia
GET/POST	/api/usuarios/	UsuarioAdminViewSet	JWT	PuedeAdministrar	Listar/crear usuarios
GET/PATCH/DELETE	/api/usuarios/{id}/	UsuarioAdminViewSet	JWT	PuedeAdministrar	Detalle/edición/desactivación (soft-delete)
POST	/api/usuarios/{id}/reset_password/	action	JWT	PuedeAdministrar	Reset de contraseña de usuario
POST	/api/usuarios/{id}/activar / /desactivar/	action	JWT	PuedeAdministrar	Activar/desactivar usuario
GET	/api/usuarios/perfiles/	action	JWT	PuedeAdministrar	Listar perfiles disponibles

6.3 apps.victimas — /api/victimas/

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 13 de 24

Método	URL	Vista	Permiso	Finalidad
POST	/api/victimas/buscar/	BuscarVictimaView	PuedeBuscarRNI (throttle busqueda_rni)	Búsqueda de víctima por documento
POST	/api/victimas/consultar-fuente/	ConsultarFuenteView	PuedeBuscarRNI	Consulta a fuente externa (mock)
GET	/api/victimas/precarga/	PrecargaOfflineView	PuedeCaracterizar	Precarga de datos para uso offline
GET	/api/victimas/padron/version/	PadronVersionView	PuedeCaracterizar	Versión del padrón offline (ETag)
GET	/api/victimas/padron/download/	PadronDownloadView	PuedeCaracterizar	Descarga del padrón offline
GET	/api/victimas/grupo-familiar/{cons_persona}/	GrupoFamiliarView	PuedeCaracterizar	Grupo familiar de una víctima
POST	/api/victimas/registrar-desde-fuente/	RegistrarDesdeFuenteView	PuedeBuscarRNI	Alta/insert de víctima desde fuente
GET	/api/victimas/{id}/	VictimaViewSet (retrieve)	PuedeCaracterizar	Detalle de víctima (con PII)

Nota: la fuente de datos detrás de estos endpoints es siempre `MockVictimaRepository` (ver sección 9), dato ficticio, no producción real.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 14 de 24

6.4 apps.formulario — /api/formulario/

Método	URL	Vista	Permiso	Finalidad
GET	/api/formulario/instrumentos/{id}/	InstrumentoViewSet	IsAuthenticated	Listado/detalle de instrumentos de encuesta
GET	/api/formulario/capitulos/{id}/	CapituloViewSet	IsAuthenticated	Capítulos de un instrumento
GET	/api/formulario/preguntas/{id}/	PreguntaViewSet	IsAuthenticated	Preguntas
GET	/api/formulario/instrumento/{codigo}/	InstrumentoCompletoView	IsAuthenticated	Instrumento completo (para uso offline)
POST	/api/formulario/evaluar-skip-logic/	EvaluarSkipLogicView	IsAuthenticated	Motor de evaluación de reglas de flujo

6.5 apps.hogares — /api/hogares/

Método	URL	Vista/acción	Permiso	Finalidad
GET/POST	/api/hogares/	list/create	PuedeConsultarOperacion	Listado/creación de hogares
GET/PUT/PATCH/DELETE	/api/hogares/{id}/	retrieve/update/destroy	PuedeConsultarOperacion	CRUD hogar
POST	/api/hogares/{id}/agregar-miembro/	action	ídem	Agregar miembro al hogar
GET	/api/hogares/{id}/miembros/	action	ídem	Listar miembros
POST	/api/hogares/{id}/subir-constancia/	action (multipart)	ídem	Subir constancia de

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 15 de 24

Método	URL	Vista/acción	Permiso	Finalidad
				tutor/cuidador
PATCH	/api/hogares/{id}/cambiar-autorizado/	action	ídem	Cambiar autorizado del hogar

6.6 apps.encuestas — /api/encuestas/

Método	URL	Vista/acción	Permiso	Finalidad
GET/POST	/api/encuestas/	list/create	PuedeConsultarOperacion	Sesiones de encuesta
GET/PATCH	/api/encuestas/{id}/	retrieve/partial_update	ídem	Detalle/actualización de sesión
POST	/api/encuestas/{id}/responder/	action	ídem	Responder una pregunta
POST	/api/encuestas/{id}/responder-bulk/	action (máx. 2000 items, transaccional)	ídem	Respuesta masiva
POST	/api/encuestas/{id}/finalizar/	action	ídem	Finalizar sesión
GET	/api/encuestas/{id}/respuestas/	action	ídem	Listar respuestas

6.7 apps.parametricas — /api/parametricas/ (solo lectura, IsAuthenticated)

Departamentos, Municipios (+ acción /todos/ sin paginar), Veredas, Tipos de Documento, Comunidades Negras, Resguardos Indígenas, Direcciones Territoriales, Puntos de Atención — cada uno con list/retrieve estándar de ReadOnlyViewSet.

6.8 apps.ia — /api/ia/

Método	URL	Vista	Permiso	Finalidad
POST	/api/ia/consentimiento/	ConsentimientoIAView	PuedeCaracterizar	Registro de consentimiento para uso de IA

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 16 de 24

Método	URL	Vista	Permiso	Finalidad
GET	/api/ia/estado/	EstadoIAView	PuedeCaracterizar	Estado de IA para una sesión
POST	/api/ia/mapear-audio/	MapearAudioView	PuedeCaracterizar (throttle ia_consulta)	Mapeo de audio a campo de formulario (Gemini), exige consentimiento previo
POST	/api/ia/procesar-entrevista/	ProcesarEntrevistaView	PuedeCaracterizar	Procesamiento batch de entrevista

6.9 apps.auditoria — /api/auditoria/

Método	URL	Vista	Permiso	Finalidad
GET	/api/auditoria/logs/	LogAccesoListView	administrar O ver_reportes	Consulta de bitácora de acceso (filtros por acción, resultado, usuario, fechas)

6.10 apps.movil — /api/movil/

Método	URL	Vista	Auth	Finalidad
GET	/api/movil/version/	version	AllowAnonymous	Última versión de APK disponible
GET	/api/movil/descargar/	descargar	AllowAnonymous	Descarga de APK (registra evento en auditoría)

6.11 apps.reportes — /api/reportes/

Método	URL	Vista	Permiso	Finalidad
GET	/api/reportes/produccion/ (alias /encuestador/)	produccion_resumen	PuedeConsultarOperacion	Resumen de

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 17 de 24

Método	URL	Vista	Permiso	Finalidad
				producción del encuestador
GET	/api/reportes/produccion/detalle/ (alias /encuestador/detalle/)	produccion_detalle	ídem	Detalle paginado
GET	/api/reportes/produccion/export/ (alias /encuestador/exportar/)	produccion_export_csv	ídem	Exportación CSV (streaming)
GET	/api/reportes/supervisor/	supervisor_reporte	PuedeVerReportes	Reporte agregado para supervisión
GET	/api/reportes/dashboard/series/	dashboard_series	PuedeVerReportes	Series temporales para dashboard

6.12 apps.sincronizacion

No expone endpoints. App registrada sin implementación (ver sección 9).

7. SEGURIDAD OBSERVABLE

Únicamente hallazgos verificables en código, sin especulación:

Controles presentes:

- Autenticación JWT con rotación de refresh token y blacklist tras rotación (rest_framework_simplejwt, base.py:227-237).
- Bloqueo de cuenta anti-fuerza-bruta en login (5 intentos / 15 min) implementado en LoginView sobre caché/Redis.
- Throttling diferenciado por endpoint sensible (login, busqueda_rni, ia_consulta).

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 18 de 24

- Cifrado de campos PII (nombres, documento, fecha de nacimiento) con Fernet (AES + HMAC) y hash SHA-256 separado para búsqueda sin exponer el dato cifrado (`apps/victimas/fields.py`).
- Contraseñas con Argon2 como hasher primario, longitud mínima 10.
- Permiso por defecto de DRF es `IsAuthenticated` (todo cerrado salvo excepción explícita) — postura "deny by default" correcta.
- Cabeceras de seguridad activas en producción: `SECURE_SSL_REDIRECT`, `HSTS` con `includeSubDomains+preload`, `X_FRAME_OPTIONS=DENY`, `cookies Secure+HttpOnly` (`production.py:56-83`).
- CORS restringido por variable de entorno en producción, con fail-safe a "ningún origen permitido" si la variable no se configura (`production.py:81-83`).
- Log de acceso (`LogAcceso`) con inmutabilidad forzada a nivel de modelo: `save()` rechaza actualizaciones y `delete()` siempre lanza excepción (`apps/auditoria/models.py:105-117`).
- Tokens del frontend en `sessionStorage` (no `localStorage`), decisión explícita documentada en el código para reducir superficie de robo de token vía XSS persistente.
- Secretos de base de datos en Docker gestionados vía Docker secrets (archivos), no variables de entorno planas, en `docker-compose.yml`.

Puntos que requieren atención (sin clasificar como vulnerabilidad confirmada sin más contexto operativo):

- `srni/settings/servidor.py` desactiva `SECURE_SSL_REDIRECT`, cookies seguras y `HSTS` heredados de `production.py` — es un archivo de configuración vivo en el repositorio; si se selecciona por error como `DJANGO_SETTINGS_MODULE` en un despliegue con datos reales, el tráfico quedaría sin forzado de HTTPS. Documentado en el propio código como fase temporal.
- CSP (Content Security Policy) está documentada solo como comentario para configurar en Nginx (`production.py:69-78`), pero **no se aplica desde Django** — no se verificó si `infra/nginx/nginx.conf` la implementa (fuera del alcance detallado de esta revisión, ver sección 9).
- Credenciales de prueba en texto plano dentro de scripts del repositorio (`srni-backend/scripts/e2e_flujo_completo.py` y otros dos scripts), usuario y contraseña de un entorno de pruebas.
- El comentario de `.env.example:15` describe el cifrado PII como "AES-256"; la implementación real usa Fernet, que internamente es AES-128 en modo CBC + HMAC-SHA256 (la clave de 32 bytes es correcta para Fernet, pero la descripción del algoritmo en el comentario no corresponde exactamente a "AES-256"). No representa una debilidad criptográfica per se (Fernet es un estándar aceptado), pero sí una imprecisión de documentación que podría confundir a un auditor de seguridad futuro.
- No se identificó SDK de monitoreo/alertamiento de errores en producción (tipo Sentry); el logging queda en `archivo/stdout`.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 19 de 24

8. ORGANIZACIÓN DEL PROYECTO

srni-backend/	Django REST Framework
├ apps/	11 apps de dominio (una vacía: sincronizacion)
├ srni/	configuración (settings/, urls.py, celery.py, asgi/wsgi)
├ scripts/	~20 scripts de generación/parcheo de fixtures y QA manual
├ tests/	suite pytest a nivel de proyecto
└ data/	CSV de municipios DANE
srni-frontend/	React + Vite (SPA administrativa/operativa)
├ src/{api, components, pages, stores, test}	
srni-mobile/	App móvil (Expo/React Native) – fuera de alcance de este informe
infra/	nginx, postgres, scripts de despliegue
docs/	documentación extensa del proyecto (arquitectura, sprints, instrumentos, seguridad)
informes/	informes mensuales de gestión (mayo-julio 2026)
docker-compose.yml	orquestración de nginx, backend, celery, postgres, redis

La carpeta docs/ es notablemente extensa y contiene documentación propia del equipo (arquitectura, modelos de base de datos, bitácoras, manuales de usuario, análisis de seguridad previos) — se recomienda contrastar el contenido de docs/backend/seguridad.md y docs/arquitectura/ARQUITECTURA.md con los hallazgos de este informe en una revisión posterior, ya que pueden contener contexto adicional sobre decisiones (por ejemplo, el estado del repositorio Mock de víctimas) que no es visible solo desde el código.

Semaforización Organización: ● Bajo

Justificación: la separación entre backend, frontend, móvil, infraestructura y documentación es clara y consistente con convenciones estándar de la industria.

9. OBSERVACIONES TÉCNICAS

- Repositorio de víctimas siempre en modo Mock.** settings.VICTIMA_REPOSITORY no está definido en base.py, development.py, production.py ni servidor.py. La función get_repository() (apps/victimas/repository/__init__.py) cae siempre a MockVictimaRepository, que contiene ~11 casos de prueba hardcoded. No existe implementación de un repositorio real (Oracle/RUV) en el código — está marcada como pendiente (TODO) en repository/base.py. **Este es el hallazgo con mayor impacto potencial del informe** y debe confirmarse con el equipo técnico si corresponde al estado esperado de la fase actual del proyecto.
- App sincronizacion sin implementación.** Registrada en INSTALLED_APPS pero sus archivos models.py, views.py, urls.py, serializers.py, admin.py están vacíos, y no tiene include() en las URLs raíz.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 20 de 24

3. **Celery configurado en infraestructura sin tareas en código.** `docker-compose.yml` define un servicio `celery` que arranca un worker sobre las colas `sync`, `reports`, y `srni/celery.py` inicializa la app Celery, pero no se encontró ningún archivo `tasks.py` ni uso de `@shared_task/@app.task` en ninguna app, ni configuración de `CELERY_BROKER_URL` en los settings. El worker, de ejecutarse, no tendría tareas que procesar.
4. **HogarValidator no conectado a las vistas HTTP.** Las reglas de negocio (tutor requiere menores, cuidador requiere dependiente, validación de autorizado) existen y tienen tests unitarios directos, pero no se detectó su invocación desde `apps/hogares/views.py`.
5. **Duplicación de lógica de extracción de IP** en 6 archivos distintos de vistas, en lugar de un middleware o utilidad compartida — no es un error funcional, pero es deuda técnica de mantenibilidad.
6. **Dependencias declaradas sin uso:** `django-auditlog` y `django-ratelimit` en `requirements.txt` sin referencia alguna en el código.
7. **Adopción parcial de librerías de formulario en frontend.** React Hook Form + Zod instalados pero usados en una sola pantalla (`CambiarPassword`); el resto de formularios usa manejo manual de estado.
8. **Cobertura de tests desigual.** El backend tiene una suite de tests razonablemente amplia (por app y a nivel de proyecto); el frontend tiene cobertura mínima (un store y un componente).
9. **Credenciales de prueba en texto plano** en tres scripts del backend (`scripts/e2e_flujo_completo.py` y otros dos), pertenecientes aparentemente a un entorno de pruebas, no de producción — de todas formas, es una práctica a corregir en el marco de higiene de repositorio.

10. CONCLUSIÓN

El proyecto presenta una base de código backend y frontend organizada, modular y con prácticas de seguridad relevantes para el tipo de datos que maneja (información de víctimas del conflicto armado, sujeta a cifrado y control de acceso). La integración entre frontend y backend es coherente y verificable en ambos extremos (autenticación JWT con refresco automático, consumo de endpoints documentado, manejo de errores centralizado).

El hallazgo que más requiere seguimiento por parte del equipo técnico y del supervisor del proyecto es la ausencia de configuración del repositorio real de víctimas (`VICTIMA_REPOSITORY`), lo que implica que el sistema, tal como está configurado en el repositorio, opera exclusivamente sobre datos simulados en cualquier entorno, incluida la configuración de producción. El resto de hallazgos (app `sincronizacion` vacía, Celery sin tareas, dependencias sin uso, validador de hogares no conectado, cobertura de tests frontend mínima) corresponden a deuda técnica o elementos pendientes de completar, sin evidencia de riesgo crítico de seguridad activo en el código auditado.

No se identificaron elementos que ameriten clasificación ● Crítico en ninguna de las secciones evaluadas, bajo la evidencia disponible en el código fuente al momento de este análisis.

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 21 de 24

Fin del informe. Análisis realizado en modo exclusivo de lectura, sin modificaciones al repositorio

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 22 de 24

6. DOCUMENTOS DE REFERENCIA

1. Ley 1448 de 2011 (Ley de Víctimas y Restitución de Tierras).
2. Manual de Imagen Institucional de la Unidad para las Víctimas.
3. docs/arquitectura/ARQUITECTURA.md — documentación de arquitectura del repositorio srni-backend/.
4. docs/backend/seguridad.md — análisis de seguridad previo documentado por el equipo de desarrollo.
5. requirements.txt y docker-compose.yml del repositorio auditado.
6. Informes mensuales de gestión del proyecto (carpeta informes/, mayo–julio 2026).

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 23 de 24

7. ANEXOS

Anexo 1. Alcance detallado de aplicaciones Django auditadas (ver numeral 3.3 de la sección Desarrollo).

Anexo 2. Inventario de endpoints por aplicación (ver numeral 6 de la sección Desarrollo).

 Unidad para las Víctimas	MANUAL, LINEAMIENTO O METODOLOGIA	Código:001B
	QA DIAGNÓSTICO	Versión: 1
	REVISIÓN BACKEND DE APLICACIÓN APK	Fecha: julio de 2026 Página 24 de 24

8. CONTROL DE CAMBIOS

Se incluye una casilla por cada versión con la descripción de las actividades de los cambios efectuados al documento.

Versión	Fecha	Descripción de la modificación
1	15/07/2026	Elaboración y emisión del informe de auditoría técnica inicial (QA Diagnóstico – Revisión Backend de Aplicación APK).